SAECOIN

CODING STANDARDS

Version 1.0

Project: SAECOIN
Ticker: SAC (Provisional)
Primary Language: Rust
Android: Kotlin + Jetpack Compose
Desktop: Tauri + TypeScript
Explorer: React + TypeScript
Repository: saecoin
Status: Planning

---

1. PURPOSE

This document defines the coding standards and development rules for SAECOIN.

All developers, contributors, and AI coding agents must follow these standards.

The objectives are:

1. Maintain readable and consistent code.
2. Reduce bugs and security vulnerabilities.
3. Protect consensus-critical functionality.
4. Avoid duplicated implementations.
5. Keep modules independent.
6. Make testing mandatory.
7. Ensure maintainability.
8. Make AI-assisted development safer and more predictable.

2. GENERAL CODING PRINCIPLES

All source code must follow these principles:

- Readability over unnecessary complexity.
- Explicit behavior over hidden behavior.
- Small modules with clear responsibilities.
- Reusable code where appropriate.
- Strong input validation.
- Safe error handling.
- Automated testing.
- Documented public interfaces.
- Minimal dependencies.
- No unnecessary code duplication.

Developers must not introduce functionality that is outside the approved project requirements without documenting and reviewing the change.

3. LANGUAGE-SPECIFIC STANDARDS

3.1 Rust

Rust is the primary language for:

- Blockchain core.
- Consensus.
- Full node.
- Transactions.
- UTXO.
- P2P networking.
- Mining core.
- CPU miner.
- GPU integration.
- Wallet core.
- API backend.

Naming

Element	Convention	Example
Crates	kebab-case	saecoin-consensus
Modules	snake_case	block_validation
Files	snake_case	transaction.rs
Functions	snake_case	validate_block
Variables	snake_case	block_height
Types	PascalCase	BlockHeader
Traits	PascalCase	BlockValidator
Constants	SCREAMING_SNAKE_CASE	MAX_BLOCK_SIZE
Lifetimes	Short lowercase	'a

Rust rules

1. Use "cargo fmt" before committing.
2. Use "cargo clippy" to identify common problems.
3. Avoid unnecessary "unsafe" code.
4. Document every justified unsafe block.
5. Avoid unnecessary cloning.
6. Prefer explicit error types.
7. Avoid silently ignoring errors.
8. Avoid panic-based control flow in production code.
9. Use checked arithmetic for monetary and consensus-critical calculations.
10. Keep public interfaces documented.
11. Avoid unnecessary global mutable state.

12. Use dependency versions deliberately.

Error handling

Production code should use structured error handling.

Do not silently discard errors using patterns such as:

let _ = operation();

unless ignoring that result is explicitly safe and documented.

For consensus-critical operations, errors must be handled deterministically and consistently.

Money and arithmetic

- Never use floating-point numbers for SAC amounts.
- Use integer smallest units.
- Check arithmetic overflow.
- Validate input and output sums.
- Document rounding behavior.
- Avoid implicit numeric conversions.

4. CONSENSUS-CRITICAL CODE RULES

Consensus code requires stricter standards than ordinary application code.

Consensus-critical modules include:

- Block validation.
- Transaction validation.
- Proof-of-work verification.
- Difficulty adjustment.
- Block subsidy.
- Supply calculation.
- Transaction serialization.
- Block serialization.
- Chain selection.
- UTXO state transitions.

Mandatory rules:

1. No undocumented consensus behavior.
2. No platform-dependent consensus calculations.
3. No floating-point consensus calculations.
4. No unreviewed consensus changes.
5. No dependency update that changes consensus behavior without review.
6. Every consensus rule must have tests.

7. Important rules must have deterministic test vectors.
8. Invalid data must be rejected consistently.
9. All nodes must follow identical validation rules.
10. Consensus changes must be documented before implementation.

Any consensus-critical modification requires:

- Written explanation.
- Related specification update.
- Test updates.
- Code review.
- Regression testing.
- Compatibility impact assessment.

Consensus changes must never be made casually to fix a local node problem.

5. KOTLIN AND ANDROID STANDARDS

Android wallet technology:

- Kotlin.
- Jetpack Compose.
- Android SDK.

Naming

Element| Convention
Classes| PascalCase
Functions| camelCase
Variables| camelCase
Constants| UPPER_SNAKE_CASE
Packages| lowercase

Android rules

1. Use Jetpack Compose for UI.
2. Separate UI and business logic.
3. Use ViewModel for screen state.
4. Keep private key operations out of UI components.
5. Never log seed phrases or private keys.
6. Handle lifecycle changes safely.
7. Handle network failures.
8. Validate user inputs.
9. Use secure storage for sensitive wallet data.
10. Test wallet backup and recovery.

The Android application must reuse shared wallet logic where practical rather than independently implementing consensus-sensitive operations.

6. TYPESCRIPT AND REACT STANDARDS

TypeScript is used for:

- Desktop wallet frontend.
- Explorer frontend.
- UI-related application logic.

Naming

Element	Convention
Components	PascalCase
Functions	camelCase
Variables	camelCase
Constants	UPPER_SNAKE_CASE
Types	PascalCase
Interfaces	PascalCase
Files	Follow framework conventions

Rules

1. Enable TypeScript strict mode.
2. Avoid unnecessary "any".
3. Validate external API data.
4. Handle loading and error states.
5. Keep components small.
6. Separate API logic from UI components.
7. Never store private keys in browser local storage without an approved security design.
8. Never expose secrets in frontend code.
9. Do not trust API responses without validation.
10. Keep UI state separate from blockchain consensus logic.

7. PROJECT ARCHITECTURE RULES

The repository must maintain clear separation between:

- Blockchain core.
- Consensus.
- Storage.
- Networking.
- Mining.
- Wallet core.
- Applications.
- APIs.
- Explorer.

Rules:

1. UI code must not implement authoritative consensus rules.
2. Explorer code must not modify blockchain state.
3. Mining software must not be trusted to validate its own submitted blocks.
4. The full node must independently validate blocks.
5. Wallet applications must use approved transaction and signing logic.
6. API services must not bypass node validation.
7. Database access must be controlled through defined interfaces.
8. Avoid circular dependencies.
9. Keep optional services separate from the consensus core.

8. SECURITY CODING STANDARDS

Security must be considered during implementation, not added only before release.

Mandatory requirements:

- Validate all external input.
- Avoid unsafe deserialization.
- Use approved cryptographic libraries.
- Never implement custom cryptography without a documented security review.
- Never hardcode secrets.
- Never log sensitive wallet information.
- Use secure randomness for cryptographic key generation.
- Protect sensitive local files.
- Handle authentication and authorization correctly.
- Apply appropriate request limits.
- Avoid leaking internal errors to public APIs.
- Audit third-party dependencies.

Security-sensitive changes require additional review.

9. DATABASE STANDARDS

Database operations must follow these rules:

1. Use explicit schemas and data formats.
2. Keep database keys documented.
3. Use atomic operations for critical state changes.
4. Handle interrupted writes safely.
5. Test crash recovery.
6. Test reindexing.
7. Test chain reorganization.
8. Never assume database data is valid without verification.
9. Keep consensus data separate from optional explorer indexes.
10. Document database format changes.

Database migrations must include a compatibility and recovery plan.

10. API CODING STANDARDS

API implementations must follow the approved API specification.

Rules:

- Use versioned endpoints.
- Validate request bodies.
- Return structured errors.
- Apply rate limits.
- Apply authentication where required.
- Avoid exposing sensitive internal information.
- Set request size limits.
- Document response schemas.
- Keep API behavior consistent.
- Use HTTPS for production remote connections.

API changes must include corresponding tests and documentation updates.

11. TESTING REQUIREMENTS

Every new feature must include appropriate tests.

Minimum expectations:

Code type	Required tests
Basic utility	Unit tests
Transaction logic	Unit and integration tests
Consensus logic	Unit, integration and test-vector tests
Database logic	Integration and recovery tests
P2P networking	Integration and malformed-input tests
Wallet logic	Unit, integration and security tests
API	Request and response tests
UI	Relevant UI and workflow tests

Tests must verify both valid and invalid cases.

A feature is not complete simply because its code compiles.

12. DOCUMENTATION STANDARDS

Every important module should contain a README or appropriate documentation.

Documentation should explain:

- Purpose.
- Responsibilities.

- Public interfaces.
- Dependencies.
- Configuration.
- Error behavior.
- Testing instructions.
- Security considerations.

Consensus-critical code must reference the relevant consensus specification.

When implementation differs from documentation, the discrepancy must be resolved before approval.

13. GIT AND COMMIT STANDARDS

Use meaningful commit messages.

Suggested format:

"type(scope): description"

Examples:

feat(consensus): implement block validation
fix(wallet): correct transaction fee calculation
test(utxo): add double spend tests
docs(protocol): update consensus specification
refactor(storage): improve database interface
security(p2p): validate incoming message size

Suggested commit types:

- feat
- fix
- test
- docs
- refactor
- perf
- security
- build
- chore

Rules:

1. Commit one logical change at a time.
2. Avoid committing generated build files.
3. Never commit secrets.
4. Do not push untested consensus changes.
5. Use pull requests for significant changes.

6. Require additional review for consensus-critical modifications.
7. Keep the main branch protected.

14. AI-ASSISTED DEVELOPMENT RULES

AI tools may be used to accelerate development.

However, AI-generated code must be treated as unverified until reviewed and tested.

14.1 Before generating code

The AI must:

1. Read the relevant project documents.
2. Inspect the existing repository structure.
3. Identify the correct module.
4. Check existing interfaces.
5. Avoid duplicating existing functionality.
6. Identify relevant tests.
7. Explain any unresolved technical assumptions.

14.2 During implementation

The AI must:

- Work on the requested module.
- Avoid unrelated file modifications.
- Follow the repository structure.
- Follow naming conventions.
- Avoid unnecessary dependencies.
- Avoid placeholder implementations presented as completed features.
- Avoid changing consensus parameters without approval.
- Preserve existing working functionality.

14.3 After implementation

The AI must report:

1. Files created.
2. Files modified.
3. Files removed.
4. Dependencies added.
5. Implementation summary.
6. Commands executed.
7. Build results.
8. Test results.
9. Known limitations.
10. Remaining tasks.

14.4 Prohibited AI behavior

The AI must not:

- Invent protocol rules.
- Invent cryptographic algorithms.
- Change maximum supply without approval.
- Modify block reward rules without approval.
- Introduce an unrequested third-party backend.
- Claim tests passed without running them.
- Claim a feature is complete when it is only a placeholder.
- Delete existing working code without justification.
- Hide build errors.
- Commit secrets.

14.5 Consensus-critical AI changes

For consensus-critical changes, the AI must first explain:

- What rule is being implemented.
- Which specification defines it.
- Which files will change.
- Which test vectors will verify it.
- Whether compatibility is affected.

Implementation should proceed only after the change has been reviewed and approved.

15. CODE REVIEW CHECKLIST

Before approving a significant change, reviewers should verify:

- Does the code follow the specification?
- Is the correct module being modified?
- Are inputs validated?
- Are errors handled?
- Are security risks considered?
- Are tests included?
- Are tests passing?
- Is documentation updated?
- Are dependencies justified?
- Are there unrelated changes?
- Is existing functionality preserved?

For consensus-critical changes, reviewers must additionally verify deterministic behavior and compatibility.

16. BUILD AND QUALITY CHECKS

Rust projects should use appropriate checks such as:

cargo fmt --all -- --check
cargo clippy --workspace --all-targets
cargo test --workspace
cargo check --workspace

The exact commands may be adjusted as the workspace grows.

Android, desktop, and explorer applications must also have their own build and test checks.

A successful build alone does not prove correctness or security.

17. DEPENDENCY MANAGEMENT

Before adding a dependency:

1. Check whether the functionality already exists.
2. Review the package's maintenance status.
3. Review its license.
4. Check known vulnerabilities.
5. Evaluate its security implications.
6. Avoid unnecessary dependency duplication.
7. Pin or constrain versions appropriately.
8. Test updates before merging.

Changes to cryptographic or consensus-related dependencies require special review.

18. CODE QUALITY POLICY

All code must be:

- Readable.
- Testable.
- Documented where necessary.
- Modular.
- Secure.
- Maintainable.
- Consistent with the approved architecture.

Avoid:

- Unnecessary complexity.
- Large functions with unrelated responsibilities.
- Duplicate implementations.
- Hardcoded configuration.
- Silent failures.

- Unverified assumptions.
- Unnecessary global state.

---

19. FINAL DEVELOPMENT RULE

SAECOIN development must prioritize correctness, security, and long-term maintainability.

No code should be considered production-ready solely because it compiles or passes a basic demonstration.

Every important implementation must be reviewed against its specification, tested, and documented.

Consensus-critical code requires the highest level of review because a defect can affect every node and every participant in the network.

END OF DOCUMENT